

Proof of Personhood Without a Registry

A measured zero-knowledge construction for one-human-one-action,
and the four optimisation paths that failed

Rehan Malik · Independent Cryptography Researcher · github.com/rehanmalik2008

Version 1.1 · Apache-2.0 · Reference implementation, circuits and raw measurements accompany this document

v1.1: the single epoch counter is split into `epoch_action` (rate-limit window) and `epoch_tree` (tree cadence), closing one open problem from §7.2. Zero added constraints; one added public signal (6 → 7).

Abstract

The internet has no way to check that a request comes from a distinct human. Current fixes reintroduce a central authority: a biometric registry, a government identity provider, or a hardware vendor's attestation root. This paper describes a construction that removes the registry.

A device holds a secret s and publishes $C = \text{Poseidon}(s)$. Independent attestors sign C once, and a backend appends C to an append-only Merkle tree. Every later proof of personhood proves *set membership* rather than re-verifying signatures, so signatures are checked once per credential lifetime, off the hot path. The login circuit is 4,309 R1CS constraints over BN254 and emits 7 public signals. Attester identity does not appear in it, so it cannot leak from it. Witness upkeep is a public broadcast, identical for every client, so no per-user query ever occurs.

On a Samsung Galaxy A12 (Helio P35, 8×Cortex-A53, 3,751 MB RAM) the proving step takes 477 ms with a native prover and 3,374 ms in the browser, warm medians over 20 runs. Peak prover resident memory is 361 MB, 9.6% of device RAM. Four attempts to close that gap failed, and each is reported with the measurement that ended it. The result is a deployment split: native for entry-level hardware, browser above it.

Security rests on a priced assumption, $\gamma \geq \beta \cdot m \cdot E$, rather than a boolean. The construction has not been audited. Section 6 states what it does not protect; Section 8 states what would refute it.

1. The problem

A server receiving a request cannot tell whether a human or a program sent it. Every application that allocates something per person depends on being able to tell: one signup per person, one vote per person, one airdrop per person, one review per person. When the check fails, the cheapest attack is to be many people at once. Douceur named this the Sybil attack and proved the general result: without a trusted certifying authority or a resource cost per identity, a single entity can present arbitrarily many identities [3].

CAPTCHAs were the practical answer for about twenty years, and they have stopped working. Vision and language models now solve the standard challenge families faster and more accurately than people do, and solving services buy human labour for a fraction of a cent per challenge when a model is not enough. Once a machine passes the test more reliably than a person, the test no longer measures what it was built to measure.

Three replacements are deployed or under serious proposal. All three work, and all three put back a central authority.

The biometric approach has a central operator enrol an iris or a face, store the template, and answer whether a given person has been seen before. A biometric template cannot be revoked and reissued the way a key can, and a person has only one iris, so a breach of the operator, a coercion order against it, or a decision to shut it down leaves the enrolled population with no recovery path. The operator also becomes the single party that decides who is counted as a person.

Government electronic identity, such as eIDAS 2.0 and the schemes modelled on it, gives strong assurance to anyone who holds documents. It makes participation depend on a state issuing those documents and continuing to issue them, which leaves out the undocumented and the stateless and gives a government a lever over who is allowed to act. It also inserts the state into transactions it previously could not see.

Hardware attestation through a secure element is cheap and works well, but it moves the root of trust to Apple, Google, or a chip vendor. Personhood then reduces to holding a device those firms have signed, and those firms gain a veto over who can act online.

The shared structure is that one party holds a record of who is real and verifiers query that party. The registry is the problem. The rest of this paper describes a construction with no registry of humans: there is a public set of cryptographic commitments, and no entry in it can be linked to a person, to an action, or to another entry.

1.1 What is claimed

A party proves to any verifier that it is a distinct human who has not made this claim before in this context, with:

- no central registry of people;
- no biometrics and no hardware root of trust;
- no revelation of which human;
- no linkability across contexts, by verifiers, by colluding verifiers, or by the issuer.

This is a personhood primitive. It answers "one human, once, here" and gives no answer to "which human". Not being able to identify anyone is the design goal, and it is also the boundary of what the primitive is good for.

2. Construction

The design separates a heavy operation that runs rarely from a light one that runs on every use. Verifying an attestation inside a circuit is expensive, so it is done once at enrolment and then carried forward by set membership.

Time is measured in two counters. `epoch_action` is the rate-limit window and the period over which one action per context is allowed; it is fast, for example ten minutes. `epoch_tree` is the cadence on which the membership tree publishes a new root; it is slow, for example daily. They are independent public inputs. Version 1.0 used one counter for both; splitting them lets a deployment rate-limit tightly without rebuilding the tree tightly, and costs zero circuit constraints.

2.1 Phase 1: enrolment, once per credential lifetime

1. The device generates a root secret s and computes the commitment $C = \text{Poseidon}(s)$. The secret never leaves the device.
2. k independent attesters sign C blind. Attester categories are social, institutional, temporal (a verifiable-delay-function anchor on presence), hardware (WebAuthn), and governmental eID. Hardware and eID are permitted as one input each. Neither is permitted as a root and neither is required.
3. The device proves the k signatures in a circuit of 13,099 constraints. The backend verifies that proof and appends C to a public append-only Poseidon Merkle tree of arity 8 and depth 9, capacity $8^9 = 134,217,728$ leaves.
4. The device caches its membership witness, which is 9 sibling groups.

2.2 Phase 2: proof of personhood, every use

The login circuit proves four statements, with every input private except the outputs:

- $C = \text{Poseidon}(s)$, knowledge of the secret behind a leaf;
- C is in the tree whose root was published for `epoch_tree`, opened in-circuit, so C itself is never revealed;
- $N = \text{Poseidon}(s, \text{ctx}, \text{epoch_action})$, a per-context nullifier that rotates with the rate-limit window;
- a rate-limiting-nullifier Shamir share $y = s + N \cdot \text{signalHash}$. Because N depends on `epoch_action` and not on `epoch_tree`, the RLN slot is $(\text{ctx}, \text{epoch_action})$ [6, 8].

The public output is exactly seven field elements: $[N, y, \text{root}, \text{ctx}, \text{epoch_action}, \text{epoch_tree}, \text{signalHash}]$. Five are inputs the verifier already chose or knows. Two are outputs. The verification key reports `nPublic = 7`. The verifier learns that one attested distinct human acted once in this context in this rate-limit window, and nothing else.

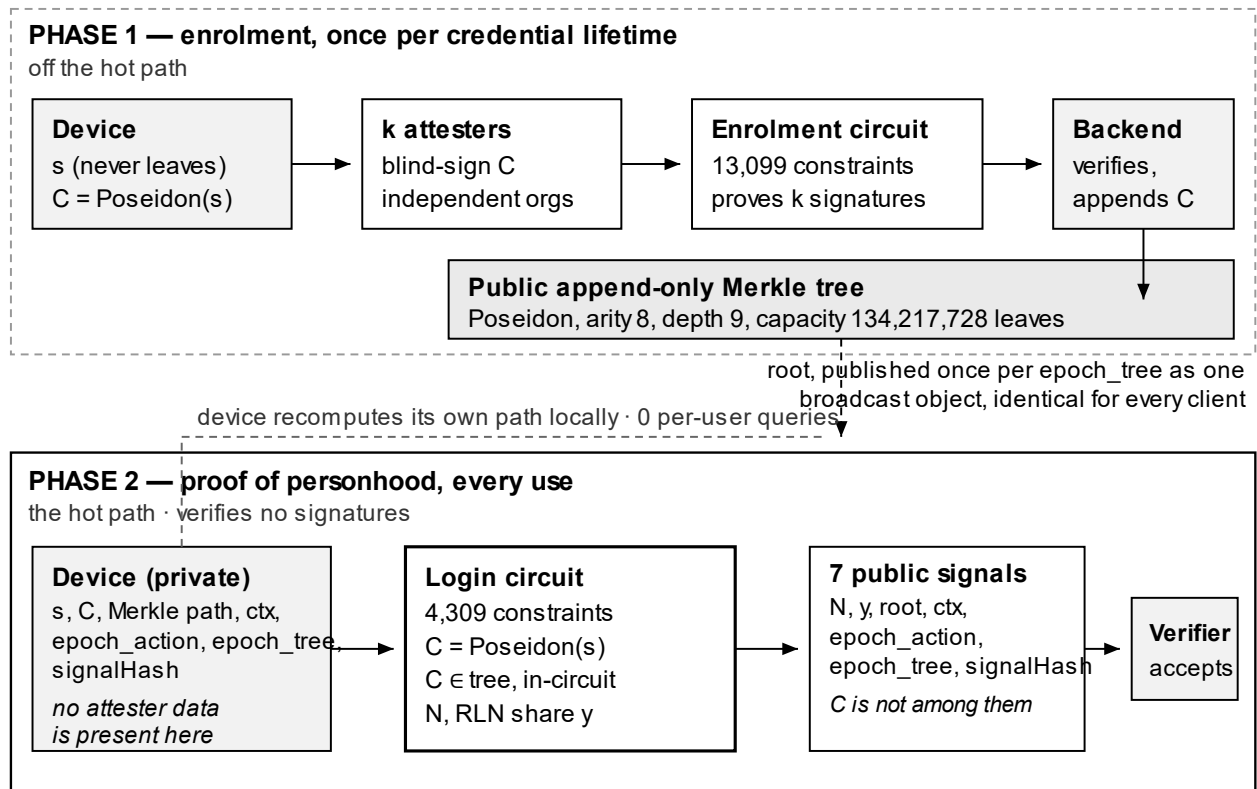


Figure 1. The two phases. Signature verification happens once, in Phase 1. Phase 2 verifies no signatures at all: it proves membership. No attester data enters the Phase-2 circuit, so none can leak from it. The nullifier binds to `epoch_action`; the tree root is the one published for `epoch_tree`.

2.3 Why membership instead of re-verifying signatures

This is the central design decision, and it was reached by measurement rather than by intent. The direct construction re-verifies k attester signatures inside the login circuit. Elliptic-curve signature verification is expensive in R1CS. One EdDSA-Poseidon verification over Baby JubJub costs about 4,207 constraints, and roughly 97% of that is two scalar multiplications, which R1CS cannot compress.

Several attempts to reduce that cost were built and measured. Halving the Merkle depth removed 7,290 constraints, and the circuit came to rest on the same signature floor. An RSA accumulator that took in-circuit membership out altogether did the same. In-circuit half-aggregation of the k signatures came out worse than verifying them directly, at 10,525 constraints against 8,890 for $k = 2$. Aggregation replaces k cheap fixed-base multiplications with one fixed-base and $k-1$ additional variable-base multiplications, and the variable-base multiplication is the costly gadget. Aggregation pays off outside a circuit because those multiplications can be batched into one; an R1CS has nothing to batch them into, so here the technique adds cost rather than removing it.

Moving signature verification out of the circuit worked, at 478 constraints, but the backend then learned which attesters had vouched. That destroys attester unlinkability, so it was rejected.

The resolution was to stop re-verifying signatures on the hot path. Attestation is a statement about a credential's origin. It only needs to be checked when the credential is created. After that, "this commitment was properly attested" is equivalent to "this commitment is in the set of properly attested commitments", which is a membership proof. Two consequences follow:

- **Cost.** Phase 2 fell from 13,099 to 4,309 constraints, a factor of 3.0. A wide, shallow Poseidon tree is what makes membership cheap: a Poseidon(W) hash costs roughly linearly in state size, but the tree needs only $\log_W(n)$ levels, so wider and shallower wins. Arity 8 at 134 million leaves costs 3,831 constraints for the opening; a binary tree at the same capacity costs 6,558, which is 71% more.
- **Privacy.** Attester unlinkability became structural rather than argued. The Phase-2 circuit contains no attester public key, no signature, and no category label. A property enforced by absence cannot be undone by a bug in a constraint.

2.4 Witness maintenance without queries

A membership proof needs a current Merkle path. The naive way to keep one current is for the client to ask the backend for its path. That request carries the client's leaf index, which is a stable identifier for that user, and it is issued on a schedule correlated with the user's activity. It would undo cross-context unlinkability outside the circuit, where the circuit cannot defend it.

The tree is append-only, so it does not need to be queried. Once per epoch_tree the backend publishes one broadcast object containing the appended frontier nodes. It is byte-identical for every client and contains no leaf index, position, or path. Each device recomputes its own affected path levels locally, 9 Poseidon(8) hashes, and asserts that the recomputed root equals the broadcast root, which also detects a divergent broadcast. No per-user request exists at any point. Because epoch_tree is decoupled from epoch_action, this republish cadence is set by tree economics, not by how tight the rate limit needs to be.

This is enforced structurally, not by policy. The server object exposes an epoch-keyed API with no method that takes a leaf index. The client refresh functions cannot reach the network and reject any broadcast carrying a position-like field. Five such guards are in place, and the test suite asserts zero upstream per-user calls across every scenario.

WITNESS REFRESH — one broadcast, every device, no queries

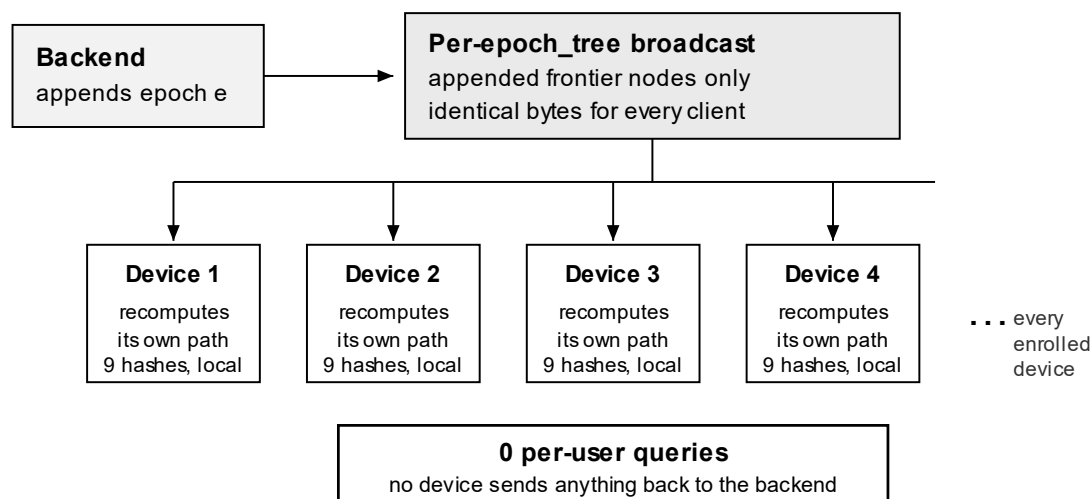


Figure 2. The per-epoch_tree broadcast is one object, byte-identical for every client, carrying no leaf index or path. Each device folds it into its own cached witness locally. Nothing flows from a device to the backend, so there is no request that could carry a stable per-user identifier.

2.5 Choosing k , and the diversity predicate

k is the number of independent attestations required at enrolment. It is a security parameter, not a constant. Setting it high raises the cost of forging a credential and also raises the friction for honest users, so it should be derived rather than assumed. With context-scoped attestation ($m = 1$) and attestation cost A per attester, the requirement of Section 4 gives $k = \lceil \beta \cdot E / A \rceil$, floored by whatever collusion tolerance the deployment wants. For low-value bot defence, $k = 2$ is often sufficient.

The diversity predicate D requires that the k attestations come from distinct categories, and in a production deployment from distinct organisations and jurisdictions. It costs 2 constraints per pair in the enrolment circuit, so it is close to free, and it is enforced in Phase 1 where the attester public keys are available. Phase 2 carries no category labels, which is why D cannot be checked there and does not need to be.

D has a structural consequence worth stating. A single organisation cannot satisfy a predicate that requires independent attesters, because its own attesters are not independent of each other. Any single-vendor deployment must either include external attesters, which means participating in a network it does not control, or drop D , which makes its variant measurably weaker. The security rule penalises centralisation directly rather than relying on governance to prevent it.

2.6 Relation to prior constructions

The components are not new. Anonymous credentials date to Chaum [1] and were made practical by Camenisch and Lysyanskaya [2]. Proving set membership in zero knowledge over a Merkle tree, with a nullifier to prevent double signalling, is the Semaphore construction [6]. The rate-limiting nullifier that burns a reused secret is Shamir secret sharing [7] applied as in [8]. Groth16 [4] and BN254 [10] are standard.

Two things here are specific. The first is moving signature verification entirely off the proving path, which is what makes the login circuit 4,309 constraints and makes attester unlinkability structural instead of argued. The second is that witness upkeep is a public broadcast with no per-user query, which closes a linkability channel that sits outside the circuit and that a circuit therefore cannot defend. Neither is a new primitive. Both are consequences of taking the end-to-end argument [5] seriously about where a property has to be enforced.

3. Security properties

Each property below is a consequence of the construction, not of an operational promise. The right-hand column states the strongest adversary the property survives.

Property	Mechanism	Strongest adversary survived
Sybil resistance	$\gamma \geq \beta \cdot m \cdot E$ (Section 4) plus the diversity predicate D	Economically rational attacker
Unforgeability	A fabricated C has no membership witness	Anyone not breaking the accumulator
Cross-context unlinkability	$N = \text{Poseidon}(s, \text{ctx}, \text{epoch_action})$; C never revealed	Colluding relying parties <i>and</i> the issuer
Attester unlinkability	No attester data exists in the Phase-2 circuit	The verifier itself
One action per context per epoch_action	Nullifier uniqueness; N binds to epoch_action, not epoch_tree	—
Double-use burn	RLN: two proofs in one (ctx, epoch_action) reveal s, for any epoch_tree	Self-enforcing; no adjudicator needed
No shared key	Membership replaces any MAC or token bridge	Removes the symmetric-key dilemma

Table 1. Security properties and the adversary each survives. Cross-context unlinkability holds against the issuer because the issuer sees only C at enrolment and only N at use, and C never appears in a Phase-2 proof.

The double-use property deserves a note because it is unusual. A rate-limiting nullifier is a degree-1 Shamir share: $y = s + N \cdot \text{signalHash}$. One proof reveals one point on a line and nothing about s. Two proofs in the same (ctx, epoch_action) with different signals reveal two points, and two points determine the line, which yields s. Losing the credential is the penalty for proving twice in one window, and it follows from the arithmetic rather than from a moderator's ruling.

The burn binds to epoch_action alone. N is $\text{Poseidon}(s, \text{ctx}, \text{epoch_action})$ with no dependence on epoch_tree, so two proofs in one rate-limit window carry the same N and lie on the same line even if they cite different tree roots. Varying epoch_tree is not a way out. If the nullifier had been bound to both counters, an attacker acting twice in one window against two different published roots would produce two distinct nullifiers and escape the burn; a test asserts the correct binding by recovering s from exactly that pair of proofs (scripts/test_rln_burn.mjs).

CROSS-CONTEXT UNLINKABILITY

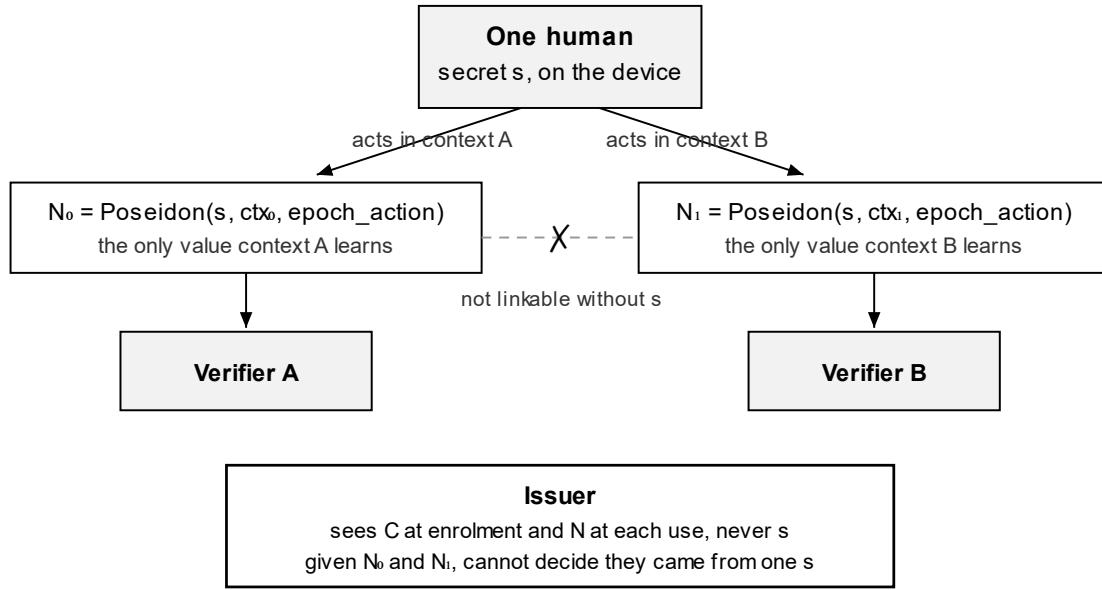


Figure 3. The privacy claim made concrete. The same s produces a different nullifier in every context. Each verifier sees only its own nullifier. Without s , no party, including the issuer that has seen both C and every N , can tell that two nullifiers belong to the same person.

4. Economics

Sybil resistance in this construction is an economic property rather than a cryptographic one. The cryptography guarantees that identities are countable and cannot be forged outright. What keeps an attacker from creating many of them is that each one costs money to mint. Getting that cost wrong is the most likely way for a deployment to fail, so the requirement is stated below as arithmetic.

4.1 The requirement

Let β be the value to an attacker of one fake identity in the application being attacked. Let γ be the cost of minting one credential. The naive condition is $\gamma \geq \beta$.

That condition is wrong, and the error is expensive. A credential is not consumed by one use. If it can act in m monetisable contexts, and it survives E epochs before expiry or burn, then one credential returns $\beta \cdot m \cdot E$ to the attacker. Deterrence therefore requires:

$$\gamma \geq \beta \cdot m \cdot E$$

A concrete case shows the size of the mistake. Suppose β is \$0.50, a credential reaches 20 monetisable contexts, and it lives 50 epochs. Then $\beta \cdot m \cdot E$ is \$500 while β is \$0.50. Pricing the mint at $\gamma = \beta$ underprices the defence by three orders of magnitude. The system would look secure and would not be.

4.2 Two ways to satisfy it, and why one is much better

The inequality can be met from either side. Raising γ to cover the widest reachable context set is simple. At $\beta \cdot m \cdot E = \500 it pushes the cost of being a person into the hundreds of dollars, which excludes the humans the system exists to serve. Result 1's own failure mode applies: if the price of being human is too high, you exclude humanity.

Shrinking m is better. Bind the attestation to a context class, so one credential cannot spray across unbounded contexts. Then $m = 1$ and the requirement becomes $\gamma \geq \beta \cdot E$. For bot defence, where β is measured in cents, the required γ lands in the range of about \$0.05 to \$2.00. That is cheap enough to be tolerable for honest users and expensive enough to make bulk forgery unprofitable. The cost is portability: a context-scoped credential is less convenient than one that works everywhere. That is a real trade and it should be stated to users rather than hidden.

4.3 Personhood is a dial, not a boolean

The consequence is that no single global answer to "is this a human?" exists. Each verifier faces a different β . A comment box and a financial account are not defending against the same attacker and should not buy the same amount of deterrence. Each verifier sets its own k , its own stake requirement, and therefore its own γ , to clear its own β .

Any system that returns `human: true` with no price attached is reporting a boolean where the underlying quantity is continuous. It is claiming more than it can support. The construction here exposes the dial rather than hiding it.

One caveat on the economics is that β has never been measured for a real application in this work. The ranges above are estimates. Section 8 lists this as a falsification criterion.

5. Measured results

Every number in this section was measured on the hardware named with it. Numbers derived by extrapolation are labelled "projected".

The reference device is a Samsung Galaxy A12 (model SM-A125F): MediaTek Helio P35, eight in-order Cortex-A53 cores, 3,751 MB RAM, Android 12, Chrome 102, retail price around \$110. It was picked as a deliberate low-end target. The people who most need Sybil resistance are disproportionately on hardware like this, so a result that only held on a flagship would not reach them.

5.1 Circuit size

Circuit	Constraints	Public signals	Role
wedge_direct_k3	13,099	3 in	Phase 1, enrolment, once per credential lifetime
wedge_mem_w8_d9_split	4,309	7	Phase 2, login, every use (split epoch)
wedge_mem_w8_d9 (pre-split, v1.0)	4,309	6	constraint-identical; the benchmark measurements in §5.3 were taken on this
+ revocation, R = 64	4,373	6/7	attester-level non-membership, +1.5%
+ revocation, R = 256	4,565	6/7	+5.9%
+ revocation, R = 1024	5,333	6/7	+23.8%

Table 2. Constraint counts from `snarkjs r1cs info` on the compiled R1CS. Groth16 over BN254. Tree capacity 134,217,728 leaves. The split-epoch circuit adds **zero constraints** and one public signal (`epoch` becomes `epoch_action` plus `epoch_tree`); the revocation costs are unchanged and apply on top of either. Revocation adds exactly **+R** constraints, all cheap linear checks.

A grep of the Phase-2 compile graph for signature gadgets returns zero matches. The verification key reports `nPublic = 7`. Both are mechanical checks a reviewer can repeat.

5.2 Memory

An early concern was that the browser prover requests a 2,048 MB WebAssembly linear memory at instantiation, which a 2 to 3 GB phone might refuse. Measuring resident memory rather than the request settled it.

Condition	Peak prover RSS	Share of 3,751 MB	Tab survived
Baseline, no background load	361.5 MB	9.6%	Yes, 20/20 runs
Under 900 MB ballast (3 tabs × 300 MB)	378.8 MB	10.1%	Yes, 20/20 runs

Table 3. Kernel VmHWM for the renderer process running the proof, captured over adb. No out-of-memory event and no lowmemorykiller activity in either condition.

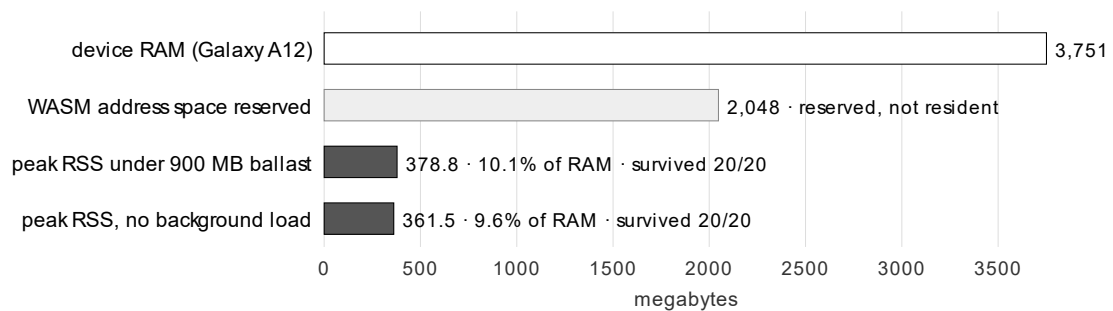


Figure 4. Memory, to scale. The reserved address space is what an early in-page measurement reported and it is 5.4× the actual resident peak. The proof fits in about one tenth of the device.

The 2,048 MB figure is reserved address space, of which about 18% is committed. Reserved memory is not resident memory. An in-page measurement using `measureUserAgentSpecificMemory` reports roughly 21 MB, which is also wrong: it misses worker-thread WebAssembly entirely. Neither figure should be used. The kernel high-water mark is the number that decides whether a tab survives.

This does not close the memory question. The specified gate requires a device with 3 GB or less. The A12 has 3,751 MB. The result is strong evidence and is not closure. It is listed as an open gate in Section 7.

5.3 Proving time

All three engines proved the same circuit from the same proving key and the same witness. Every proof was verified against the same snarkjs verification key, and the public signals were compared byte for byte and found identical. The engines are interchangeable at the protocol level.

These on-device measurements were taken on the pre-split circuit (`wedge_mem_w8_d9`, v1.0). The split circuit is constraint-identical (4,309, Table 2) and differs only by one extra public input, so the prover times carry over unchanged; on desktop, Node, the two circuits prove within run-to-run noise of each other. The snarkjs and C++-to-WASM engines were re-run on the split circuit and verify identically with the new 7-signal public output (Appendix A); rapidsnark native was not re-run because it needs the A12, and cross-engine equivalence is engine-independent.

Engine	Cold	Warm median	Warm p95	Install
snarkjs, browser, deployed today	5,187 ms	3,374 ms	—	none
C++ compiled to WASM (rapidsnark via Emscripten)	3,071 ms	2,467 ms	2,671 ms	none
rapidsnark, native	510 ms	477 ms	626 ms	app or SDK

Table 4. Proving step only, Galaxy A12, 20 runs per engine. Witness calculation in the browser adds 1,164 ms warm and 1,592 ms cold on top of the browser figures.

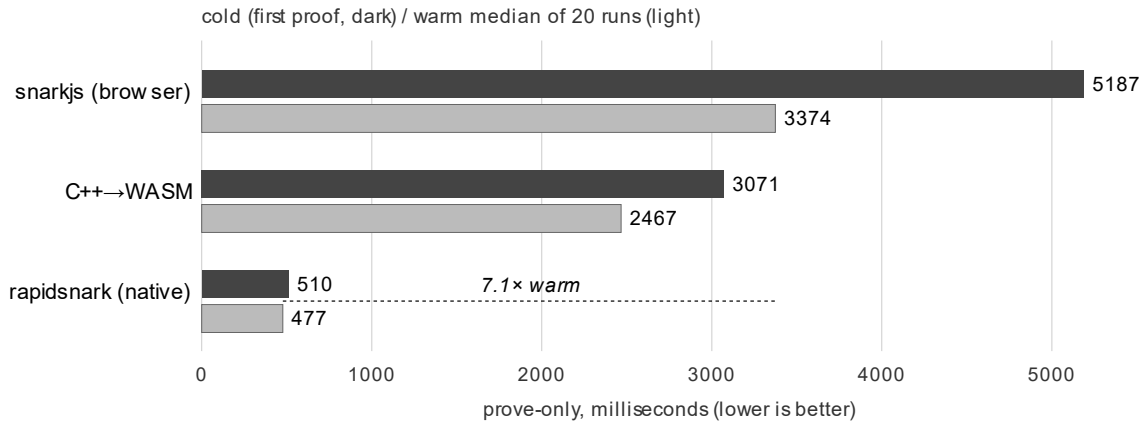


Figure 5. The same data. All three engines prove the same statement from the same proving key, and all three proofs verify against the same verification key with identical public signals (Appendix A).

The native prover runs 7.1 times faster than the browser prover on the warm median and 10.2 times faster from cold. For the native prover the first proof after startup costs about 7% more than a warm one; for the browser it costs 54% more. A real user proves once, from cold, so the browser number that matters in practice is the larger one.

5.4 Four optimisation paths that failed

The gap between 477 ms native and 3,374 ms in the browser is the difference between a zero-install product and one requiring an app on entry-level hardware. Four ways to close it were attempted. All four failed. Each is reported with the measurement that ended it, because the reasons are more useful than the outcome.

Path 1: free browser levers. Threading was already active. Cross-origin isolation was enabled and `crossOriginIsolated` was true on the device. Measured threading speedup was $2.51\times$ (12,580 ms single-thread to 5,011 ms with 8 threads, full prove, warm median). The browser supports WebAssembly SIMD, but snarkjs 0.7.6 ships no SIMD build and does not use `SharedArrayBuffer` for the prover. There was nothing unclaimed to switch on.

Path 2: compile the C++ prover to WebAssembly. rapidsnark was compiled with Emscripten, using pthreads, SIMD, and mini-gmp with 64-bit limbs in place of GMP. The build works and its proofs verify against the same verification key with identical public signals. It measured 2,467 ms warm, which is $5.17\times$ the native 477 ms. The projection for this path was $1.25\times$ to $2\times$ of native. The gate was set in advance at $2.5\times$, so the result failed by a factor of two. The build is $1.37\times$ faster than snarkjs on the proving step and $2.2\times$ faster cold, which is a real gain and is not enough.

Path 3: hand-written SIMD field arithmetic. Before writing any, the 5× was attributed. The prover's thread-pool size was made settable at runtime, and both engines were measured at matched thread counts. Native was constrained with taskset.

Threads	Native (ARM64 assembly)	WASM (portable C++)	Ratio
1	2,743 ms	13,319 ms	4.86×
2	1,442 ms	7,684 ms	5.33×
4	799 ms	4,229 ms	5.29×
8	518 ms	2,559 ms	4.94×

Table 5. Matched-thread comparison, Galaxy A12, same proving key and witness. Scaling from 1 to 8 threads is 5.30× for native and 5.20× for WASM. The single-thread WASM figure comes from a build with no pthreads and no memory growth, so it carries no threading overhead at all.

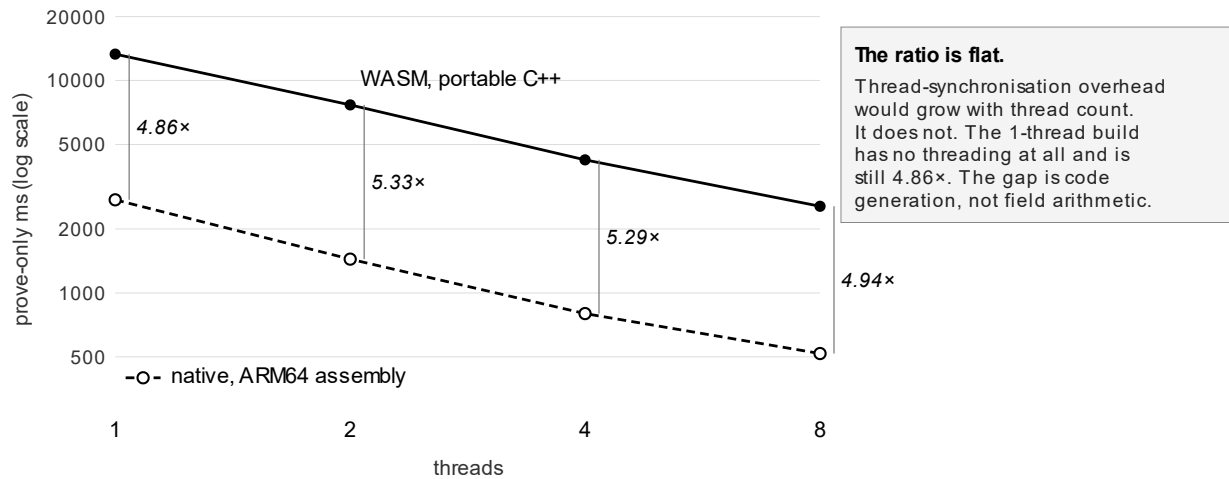


Figure 6. The measurement that ruled out writing field-arithmetic assembly. Both axes are the same circuit and the same proving key on the same handset. Log scale, so the two lines being parallel is exactly the claim: a constant factor, independent of thread count.

The ratio is flat. If the penalty were thread synchronisation, it would grow with thread count, because there would be more synchronisation to pay for. It does not grow. WebAssembly also parallelises as well as native here, 5.20× against 5.30× across the same range. A single-threaded build with every threading artefact removed is still 4.86× slower than native.

That result rules out field-arithmetic assembly. The gap is in code generation and runtime, and assembly for the field layer would target a layer that is not the bottleneck. The measurement cost one afternoon and avoided a multi-month effort aimed at the wrong place.

A second finding supports this. The best available WebAssembly field arithmetic is not SIMD. Reduced-radix scalar arithmetic with 29-bit limbs outperforms every simd128 approach in browsers, because WebAssembly lacks a 64×64→128 widening multiply and its SIMD is fixed at 128 bits. The achievable improvement over generic 64-bit CIOS is roughly 1.3× to 1.7×. The field layer therefore cannot close the gap on its own: its best available improvement is about 1.7× against a shortfall of five.

Path 4: a Rust prover with different code generation. arkworks compiled to WebAssembly was the remaining candidate. Inspection of ark-ff's Montgomery backend shows its only assembly path is gated on

x86_64 with BMI2 and ADX. For wasm32 it uses portable scalar Rust with unrolled CIOS, which is the same technique as the generic C++ path already measured, lowered through the same LLVM WebAssembly backend. Published browser figures for that stack on a Galaxy S23 Ultra, a flagship, give 793 ms for a comparable circuit. Scaled to the A12 that projects to roughly 1,900 to 2,100 ms, which is the same class as the 2,467 ms already measured. Changing the language the prover is written in does not change the LLVM WebAssembly back end that both languages compile through, and the constant factor lives in that back end.

The conclusion is that the $\sim 5\times$ belongs to the WebAssembly toolchain and browser runtime for this workload, not to this code. It should narrow as LLVM code generation improves and as relaxed SIMD and Memory64 mature. The engine is one C++ codebase, so that improvement arrives on recompilation.

5.5 Deployment matrix

Device class	Zero-install browser	Native SDK	Source
Entry-level (Galaxy A12 class, $\sim \$110$)	~ 4.7 s cold end-to-end via C++/WASM; ~ 6.8 s via snarkjs	~ 0.5 s proving step — GO	measured
Mid-range	$\sim 2\text{--}4$ s (<i>brackets the ≤ 2 s target; unmeasured</i>)	GO	projected
Desktop / flagship	~ 0.7 s full flow	GO	measured on desktop

Table 6. Deployment matrix. The circuit, proving key, nullifier construction, public signals, and privacy properties are identical in every cell. The split is a deployment choice, not a protocol fork. Desktop figure is an i7-9850H with Chrome, multi-threaded, full-flow p95 738 ms over 20 runs. The mid-range browser figure has not been measured on a mid-range device: the projected 2 to 4 s straddles the 2 s target, so on that projection some mid-range phones would clear it and some would not. Native end-to-end below ~ 1.1 s is projected; only the 477 ms proving step is measured.

The original target was 2 s for the full flow. That target assumed proving on every login. It has been revised, on the record, to 2 s on mid-range hardware and 5 s cold on entry-level hardware, proven once per epoch_action rather than once per login. Because the nullifier is scoped to epoch_action, a relying party can cache the result of a proof for that window: within one epoch_action the same human at the same relying party presents the same nullifier, so one proof covers it. The split does not force a fast epoch_action. A deployment that wants infrequent proving sets a long epoch_action and pays the several-second cost that rarely; a deployment that wants tight rate-limiting sets a short one and either re-proves that often or uses the native prover, where proving is 477 ms and the cadence does not matter. Before the split these two choices were the same knob. Deployments that must prove per login on entry-level hardware should use the native prover.

5.6 Witness maintenance at load

The tree publishes one broadcast per epoch_tree, a cadence set by tree economics alone. It is no longer tied to the rate-limit window: a deployment can rate-limit every ten minutes on epoch_action and still republish the tree once a day. The figures below are per epoch_tree, at various enrolment rates.

New enrolments per epoch_tree	Raw append log	Broadcast	Compression
1,000	31.3 KB	4.7 KB	6.6×
10,000	312.5 KB	44.9 KB	7.0×
100,000	3.1 MB	446.7 KB	7.0×

Table 7. Public broadcast size per epoch_tree, frontier-compressed. Identical bytes for every client.

Steady-state client refresh is 3 to 4 ms on desktop, projected at about 15 ms on a low-end phone, and is $O(\text{depth})$ regardless of enrolment rate. A client that goes dark for a month during a 100,000-per-epoch_tree surge downloads 13 MB and recomputes its witness in 122 ms on desktop. Upstream per-user calls during every test, including the month-offline catch-up: zero.

5.7 Verifier cost and integration surface

Verification is cheap and constant. Groth16 verification of the 7-signal proof measures about 20 ms in a browser on an i7-9850H, taken as the difference between full-flow and prove-only medians over 20 runs. Proof size is fixed at 3 group elements regardless of circuit size. The verifier does no elliptic-curve work proportional to the tree or to the number of users. On top of the zero-knowledge check it runs two constant-time freshness checks: the epoch_tree root within the last w published roots, and epoch_action within a separate, tighter recency bound. Both are deployment config; a root older than w is rejected regardless of epoch_action.

Whether a relying party can adopt this without rearchitecting was tested rather than asserted. A proof of concept runs a real OpenID Connect provider and two relying parties over real HTTP, issuing real Groth16 proofs from the wedge_mem_w8_d9_split artefacts. The nullifier is carried inside a W3C Verifiable Presentation in the ID token's vp claim, and is surfaced to the relying party as the OIDC sub. Ten assertions pass:

- first login at relying party A is allowed;
- a replay in the same epoch_action is rejected with HTTP 403;
- login at relying party B by the same human is allowed;
- sub differs between A and B, which is the pairwise, sector-scoped property;
- neither subject is a prefix, suffix, or rotation of the other;
- sub rotates when epoch_action changes;
- A allows the same human again in the new epoch_action;
- advancing epoch_tree alone (a tree republish) does not grant a new action: the same human is still rejected with 403;
- advancing epoch_tree alone does not rotate sub;
- advancing epoch_action again rotates sub and re-allows the action.

The provider generated every proof and still cannot link them, because the linkage exists only inside Poseidon(s , $\cdot$) and it does not hold s . Integration is one verification call of about 80 lines, a poll of a /roots endpoint, and a (sub, epoch_action) deduplication check. No new credential format and no custom protocol. The one semantic change a relying party must absorb is that sub is scoped to

epoch_action, so it is an action token and not an account key. The last three assertions show the decoupling directly: republishing the tree neither resets the rate limit nor rotates the subject.

5.8 Revocation, both paths

RLN stops one human from acting twice in a window. It does nothing about a credential that should no longer be valid at all: a lost device, or a compromised attester whose whole issuance batch must be undone. Two mechanisms cover the two cases, and both were built and measured against the 4,309-constraint split baseline rather than estimated.

Individual revocation, in circuit. The split circuit gains an in-circuit non-membership check of the credential commitment C against a published revocation list of R field elements. For each entry it enforces $(C - r_i) \cdot \text{inv}_i = 1$, which has no witness when $C = r_i$, so a revoked holder cannot generate a proof at all. The constraint cost is exactly $+R$, measured: 4,373, 4,565, and 5,333 constraints for R of 64, 256, and 1,024, against a 4,309 baseline. Prove time on an i7-9850H rises by 21 ms, 20 ms, and 123 ms over a 538 ms base, median of 12 rounds. The public-input count becomes $7 + R$, so the list is carried explicitly by the verifier. A production deployment would instead commit the list as one accumulator root and prove non-membership against that, moving verifier input back to $O(1)$ at a few hundred constraints per tree level; the public-list form here is the tested one.

Bulk revocation, tree rebuild. At an epoch_tree boundary the operator rebuilds with revoked leaves set to zero. Indices are not compacted, so every surviving leaf keeps its position and only the internal nodes on the revoked paths change. That makes the rebuild broadcast incremental. Measured on the arity-8 depth-9 tree, across occupancies of 10k, 50k, and 200k leaves and revocation fractions of 1, 5, and 10 percent, the tombstone broadcast ranges from 9.3 KB to 1.18 MB, and grows with the number of revoked leaves times the depth, not with tree size. Full republication of the same trees is 0.35, 1.74, and 6.98 MB, so the incremental form is 6 to 39 times smaller. An unaffected client folds the broadcast into its cached witness in 2.4 to 5.1 ms median, well under the 122 ms month-offline catch-up baseline from §5.6, and never queries the tree. A revoked client's cached witness stops hashing to the published root, its refresh throws, and a fresh proof against the new root fails because its path no longer verifies.

Nine assertions cover both paths in `scripts/test_revocation.mjs`: an honest credential proves; the same credential placed on the revocation list cannot produce a witness; after a tombstone rebuild that revokes one leaf, the revoked client's refresh throws and it cannot prove against the new root, while an unaffected client refreshes query-free and proves. The liveness cost of the bulk path is disclosed in §6: a rebuild interrupts every client for one refresh window.

6. What is not protected

This section is complete. A reader who stops here has an accurate picture of the system's limits.

The nullifier is personal data. N is a pseudonymous identifier under GDPR. The verifier learns no name, no C , and no device identifier, but N is not anonymous. Within one $(\text{ctx}, \text{epoch_action})$ a relying party has a stable handle for that user and can build a per-context profile of that user's actions. That is the intended function, because Sybil resistance requires a stable per-context identifier. Cross-context linkage remains prevented.

The nullifier is scoped to epoch_action and is not an account key. It is a per-window action token. In a new epoch_action the same user at the same relying party presents a different N. A relying party using sub as an account primary key must change that assumption. Persistent pseudonymous accounts would need a second, epoch-independent pairwise identifier, costing about 255 constraints and no new cryptography. It is not built.

Trust is not zero. The honest claim is "no single trusted party, given a 1-of-N-honest publication medium". Root equivocation is always detectable after the fact by any auditor holding the root history. It is *prevented* only under 1-of-N honesty across the client's root sources. An earlier version of this work claimed "zero trusted parties". That claim is retracted.

The 1-of-N-honest clause is now an *enforced client-side check*, not a stated assumption. Before proving, and again before accepting, the client fetches the hash-chained transparency head ($\text{head}_e = \text{Poseidon}(\text{head}_{\{e-1\}}, \text{root}_e, e)$) from N independent root sources and cross-checks. It classifies each reachable source as agreeing, lagging-but-on-the-chain (tolerated within the staleness window w), or disagreeing, and fails closed on any disagreement or below a quorum of reachable sources (default 2 of 3). This raises the adversary's requirement from "control the client's one root source" to "control every source the client queries, simultaneously and consistently." It does not defeat a full eclipse; see §7.2.

Independence of the root sources is normative. The check is worthless if the sources share a failure domain. They must differ in operator, in network path, and where possible in trust root: for example a public chain, a CDN-backed static endpoint, and a third-party bulletin board. Three endpoints on one operator's infrastructure are one source with three URLs, and would make the trust claim look stronger while changing nothing.

Timing correlation. Unlinkability holds per proof. It does not hold against traffic analysis. Witness refresh must run on a schedule independent of proving, and download sizes must be padded. These are normative requirements, not suggestions. A deployment that refreshes witnesses when a user logs in has reintroduced the correlation the broadcast design removed.

Attester collusion. k colluding independent attesters can mint fake credentials. The diversity predicate D requires attestations from organisations in different jurisdictions, which raises the coordination cost. It does not eliminate the attack. This is a sociological problem and it is not fully solvable by cryptography.

Bulk-revocation liveness. Revoking a compromised attester means rebuilding the tree without its leaves (§5.8). The rebuild is published as one broadcast and clients recompute locally, as for a normal epoch. But between the rebuild being published and a given client's next refresh, that client cannot prove: its cached witness no longer hashes to the current root. An attester compromise therefore causes a refresh-window interruption for the whole population. This is a liveness cost, not a security one, and it is the price of bulk revocation without a per-user query.

Credential rental and coercion. Anonymous credentials resist rental economically: oversubscription is unobservable to the buyer, contracts are unenforceable, and the seller can rotate freely. They are not impossible to rent. For bot defence this is acceptable, because a rented credential still consumes one real human's action budget, so renting redistributes a scarce human action rather than manufacturing bots. For defence against coordinated inauthentic behaviour it is weaker: an attacker renting from P humans obtains P coordinated genuine actions and bypasses γ entirely. Security in that setting additionally requires the

rental market to be illiquid. Human-presence gating is the sound application. Coordinated-behaviour defence carries this caveat.

State-scale adversaries. Every economic bound in Section 4 assumes an attacker constrained by solvency. An adversary who does not care about cost is not covered.

Not post-quantum. The construction uses BN254 and Groth16. A crypto-agility path exists, and migration follows the ecosystem rather than leading it.

Trusted setup. Groth16 requires a per-circuit trusted setup. A compromised setup breaks soundness, meaning forged proofs become possible. It does not break privacy. The proving keys distributed with this work come from a throwaway single-contribution setup and are for evaluation only. Production requires a public multi-party ceremony or migration to a transparent-setup proof system.

7. Open problems

7.1 Gates that block production

Five of the six remain open; all are fundable external work rather than open research, and none is a cryptographic unknown. Gate 6 is now closed (below).

#	Gate	Why it blocks	Status
1	Two or more independent circuit audits	An under-constrained signal silently breaks soundness. Reproducible builds prove the binary matches the source; they say nothing about whether the constraint system is correct.	Open. Audit-ready, not audited.
2	GDPR right-to-erasure legal memo	An append-only tree versus a deletion right. Blocks EU deployment until analysed.	Open
3	Peak RSS and p95 on a device with 3 GB or less	The formal memory gate. The A12 at 3,751 MB is evidence, not closure.	Open
4	IP indemnification and liability structure	Enterprise contracts require it.	Open
5	Trusted-setup ceremony, or migration to a transparent-setup system	Soundness assumption. Current keys are evaluation-only.	Open, disclosed
6	Attester-level revocation mechanism	RLN covers double-use only, not a compromised attester.	Closed. Both paths implemented, measured, tested (§5.8): an in-circuit non-membership check for individual revocation (+R constraints, measured +64 / +256 / +1,024), and an incremental tree rebuild for bulk revocation. Liveness cost disclosed in §6.

Table 8. Production gates. Gate 6 moved to Closed in v1.1.

7.2 Research questions that remain

Resolved since v1.0, and so removed from the list below. First, *separating the rate-limit window from the tree publication cadence*. One counter used to serve as the nullifier epoch, the rate-limit window, and the

tree cadence at once. It is now two independent public inputs, `epoch_action` and `epoch_tree`, at a cost of zero constraints. The RLN burn binds to `epoch_action` alone; a test recovers `s` from two proofs in one action window that cite different tree roots, confirming the binding (`scripts/test_rln_burn.mjs`). Second, *the bulk attester revocation trade*. It has now been measured rather than estimated (§5.8): the tombstone rebuild broadcast is incremental, 9 KB to 1.2 MB at realistic occupancy against 0.35 to 7 MB for full republication, and an unaffected client recomputes its witness in 2 to 5 ms. The periodic rebuild is the practical path; carrying every revoked commitment in `R` is not. See §7.1 Gate 6.

- **A measured β .** The economic parameter has never been measured against a live application. Every figure in Section 4 is an estimate. This is the single most valuable missing number.
- **Full eclipse.** A targeted eclipse is detectable given one honest root source, and the client now enforces that condition: before proving and before accepting, it cross-checks the hash-chained transparency head across `N` independent root sources and fails closed on any disagreement or below a quorum of reachable sources (§6). This raises the bar from controlling the client's one root source to controlling every source it queries, at once and consistently. It does not defeat a full eclipse. An adversary who holds all `N` sources simultaneously still passes the check, and that case is only auditable after the fact.
- **Diversity in practice.** Whether 3 to 5 genuinely independent attestors exist in any given jurisdiction is an empirical question about institutions. It has not been surveyed.
- **The WebAssembly gap.** The $\sim 5\times$ penalty is a toolchain property. Whether relaxed SIMD, Memory64, and improved code generation close it to the point where entry-level hardware needs no native prover is a question for the WebAssembly ecosystem rather than for this construction.

8. Falsification criteria

Stated in advance so that they cannot be adjusted afterwards. This work should be considered refuted if any of the following holds.

1. Credentials trade on an open market at a stable price for more than one quarter. Economic non-transferability would have failed.
2. Fewer than 3 to 5 genuinely independent attestors exist in a target jurisdiction. The diversity predicate `D` would be theatre, and the security argument that rests on it would be void.
3. The measured β for a target application exceeds the γ that honest users tolerate. Securing the application would price out the humans it is meant to admit.
4. A circuit audit finds an under-constrained signal permitting forged proofs.
5. No platform adopts even a zero-risk shadow-mode evaluation within 12 months. The value proposition would be wrong regardless of whether the cryptography is correct.

9. Status

What exists: a measured, working system that proves personhood privately, with reproducible builds, drop-in OIDC and W3C Verifiable Presentation integration, query-free witness maintenance, a closed prover-

optimisation search, and a documented deployment matrix. Proofs from three independent prover implementations verify against one verification key with identical public signals.

What does not exist: an independent audit, a production deployment, a design partner, a measured real-world β , and closure on the five gates in Section 7 that remain open. The sixth, attester-level revocation, is closed in this version.

The accurate description is a complete, measured, unreviewed engineering system with a candidate security property. It is not a validated standard. Validation is conferred by adversaries who fail to break a system and by adopters who build on it, and neither has happened yet. Every gate in Section 7.1 is ordinary work that follows a first deployment. No cryptography remains on the critical path.

9.1 Delegation to AI agents (not built, not analysed)

An attested human could issue a delegated credential to a software agent, so the agent acts with attributable human backing without claiming to be a person. The primitives already exist: nullifiers in a separate namespace for agent actions, the RLN share to bound rate, and epoch scoping to bound lifetime. None of it has been built, and none of it has been analysed.

Two requirements decide whether it is sound. First, delegation must be visible. A verifier must always be able to distinguish an agent acting under delegation from a human acting directly. If it cannot, the construction becomes the bot-laundering machine it exists to prevent. Second, a per-human cap on live delegations is required. Without one, delegation multiplies the actions a single credential can take, which makes it a Sybil amplifier and defeats the economics of Section 4.

Both are stated here as requirements, not as solved problems. This direction needs its own security analysis before it is worth building.

The circuits, the reference implementation, the benchmark harnesses, the raw measurement files behind every number in Section 5, and the chronological research record including the retractions are published under Apache-2.0 with its Section 3 patent grant.

References

1. D. Chaum. Security without identification: transaction systems to make big brother obsolete. *Communications of the ACM*, 28(10):1030–1044, 1985.
2. J. Camenisch and A. Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *EUROCRYPT 2001*, LNCS 2045, pages 93–118.
3. J. R. Douceur. The Sybil attack. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, LNCS 2429, pages 251–260, 2002.
4. J. Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, LNCS 9666, pages 305–326.
5. J. H. Saltzer, D. P. Reed, and D. D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
6. Semaphore: a zero-knowledge protocol for anonymous signalling and double-signalling prevention on Ethereum. Privacy and Scaling Explorations, Ethereum Foundation.
7. A. Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

8. Rate-Limiting Nullifier: a construction for anonymous rate limiting using Shamir secret sharing. Privacy and Scaling Explorations, Ethereum Foundation.
9. L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger. Poseidon: a new hash function for zero-knowledge proof systems. In *USENIX Security 2021*, pages 519–535.
10. P. S. L. M. Barreto and M. Naehrig. Pairing-friendly elliptic curves of prime order. In *Selected Areas in Cryptography (SAC) 2005*, LNCS 3897, pages 319–331.
11. R. C. Merkle. A digital signature based on a conventional encryption function. In *CRYPTO '87*, LNCS 293, pages 369–378.
12. E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von Neumann architecture. In *USENIX Security 2014*, pages 781–796.

Appendix A. A real proof

Claims about zero-knowledge systems are cheap. This appendix contains an actual proof, produced by running the published artefacts, and the transcript of verifying it. A reader can regenerate all of it.

A.1 The artefacts

File	Bytes	SHA-256
build/wedge_mem_w8_d9_split.r1cs	3,756,792	fbd04ecd...83eddc27
web/wedge_mem_w8_d9_split.wasm	3,212,057	f016efcf...b4def3a1
web/wedge_mem_w8_d9_split_final.zkey	5,764,987	bd7169e4...a16acc47
web/wedge_mem_w8_d9_split_vkey.json	4,023	a4428d64...faecb356

Table 9. The R1CS and WASM hashes match `supply-chain/artifacts.lock.json`, which `scripts/repro_build.mjs --double` reproduces byte-for-byte from a clean compile of `circuits/`. The proving key is not reproducible, because it carries ceremony randomness; it is verified against the R1CS instead.

A.2 The proof

Groth16 over BN254. Three group elements, 8 field coordinates, constant size regardless of the 4,309 constraints behind it. Re-running `mkproof.mjs` produces different values here and identical values in Table 10, because Groth16 proofs are randomised. Section A.5 uses that property.

```

pi_a = [
  2084570759954426533564902785562096468737422523455596290683747059733598569414,
  1567567686339441980740655973037063527616023530367543601680152750759479006181, 1 ]
pi_b = [ [20617072746101944515302261723900367908536597911312429818557596406791920386776,
  14292379644756314352710954835857644588754862533916600688054794785036211194318],
  [14391897212804863468710221759945629259770916899971487247562052198551236621147,
  5234233511664559673585493239283667660077587361521720131010362193139911735457], [1,0] ]
pi_c = [
  19831573536458535304144255033227867520349154756296659724042617945786514259466,
  11787348736065937792382751628962761115476684537051248968393278075380426303238, 1 ]
protocol = "groth16", curve = "bn128"

```

A.3 The seven public signals

#	Signal	Value
0	N, nullifier (output)	14824683023501643632115653304856446903925650282274056340992344805449552618966
1	y, RLN share (output)	6245802399600452304690568138657552093742378804302159597634167049696237901356
2	root, tree root for epoch_tree (input)	20550735135605091646947741900747264636731978524347924805314550072189850280242
3	ctx, context (input)	987654321
4	epoch_action (input)	11
5	epoch_tree (input)	7
6	signalHash (input)	143108474036684829574744961435355602015188502350286015805048154501691818351

Table 10. That is the whole disclosure. There is no eighth signal. The commitment C, the secret s, the Merkle path, and every attester identity are private witnesses with no output wire. The verification key reports `nPublic = 7`. Signal 4 is `epoch_action`, signal 5 is `epoch_tree`, and the nullifier at signal 0 is `Poseidon(s, ctx, epoch_action)` with no dependence on `epoch_tree`.

A.4 Verification transcript

VERIFY: true	TAMPERED: false	nPublic: 7	proveMs: 849	verifyMs: 15
--------------	-----------------	------------	--------------	--------------

VERIFY: true is `snarkjs.groth16.verify(vkey, publicSignals, proof)` on the proof above. TAMPERED: false is the negative control: incrementing public signal 0 by one and re-verifying the same proof. It is rejected. A verifier that accepts both is broken, so the negative control matters as much as the positive one. Timings are Node on an i7-9850H and are not the handset figures from Section 5.

A.5 Cross-engine equivalence

The split circuit was proved by two engines on desktop: `snarkjs`, and `rapidsnark` compiled to WebAssembly (the same C++ prover from Section 5.4, run in Node). Comparing the two outputs (`scripts/xengine_split_check.mjs`):

Comparison	Result	Expected
public signals, all 7, byte-for-byte	identical	yes; the statement is the same
<code>pi_a[0]</code> , <code>snarkjs</code>	218834494306333257751695454936...	—
<code>pi_a[0]</code> , <code>rapidsnark</code> →WASM	967877403452917643565710914541...	differs; Groth16 proofs are randomised
both verify against the same 7-signal vkey	true	yes

Table 11. Identical public signals with different group elements is the correct outcome. Groth16 proofs are randomised, so two proofs of the same statement differ. Equivalence is established by both verifying under one verification key against the same public signals, not by the proofs matching. The `pi_a[0]` values shown are from one run and change on every run. `rapidsnark` native (arm64) was validated this way on the pre-split circuit on the A12 and is not re-run here; the equivalence argument is engine-independent.

A.6 Regenerating this appendix

```
npm install
node scripts/build.mjs // compiles wedge_mem_w8_d9_split (Step 2); constraint counts in Table 2
node scripts/test_rln_burn.mjs // the burn invariant: 15 assertions, all green
node scripts/xengine_split_check.mjs // cross-engine equivalence, Table 11
node integration/run.mjs // OIDC + VP drop-in, 10 assertions
node paper/mkproof.mjs // the proof and transcript in A.1 to A.4
```

Reproducing the measurements. Every figure in Section 5 traces to a JSON capture in `results/` in the accompanying repository. `scripts/repro_build.mjs --double` proves the deployed prover is byte-identical to a clean compile of the published circuits. `scripts/bench_rapidsnark.mjs`, `scripts/bench_wasmprover.mjs`, `scripts/bench_attrib_native.mjs` and `scripts/bench_attrib_wasm.mjs` reproduce Tables 4 and 5 over adb on an Android device. `scripts/measure_phone_rss.mjs` reproduces Table 3. `docs/EVALUATION.md` is a walkthrough for a reviewer verifying the core claims independently.